

Ticker

A single-machine stock dashboard: live prices streamed to the browser, a portfolio derived from an append-only ledger, and an AI analyst that can propose changes but never make them. This guide covers the architecture end to end, how each layer works, and how the system was built.

LIVE TICKER-DASH.FLY.DEV BACKEND PYTHON 3.12 / FASTAPI FRONTEND REACT 18 / VITE TESTS 291
MACHINES 1 COST ~\$3/MO

PROJECT AT A GLANCE

Five layers, one process, three rented services

Everything below the browser runs as a single Python process on a single machine. That is a deliberate constraint, not a limitation of scale: the market feed allows one connection per key, and the database is a file on a disk that cannot be mounted twice.

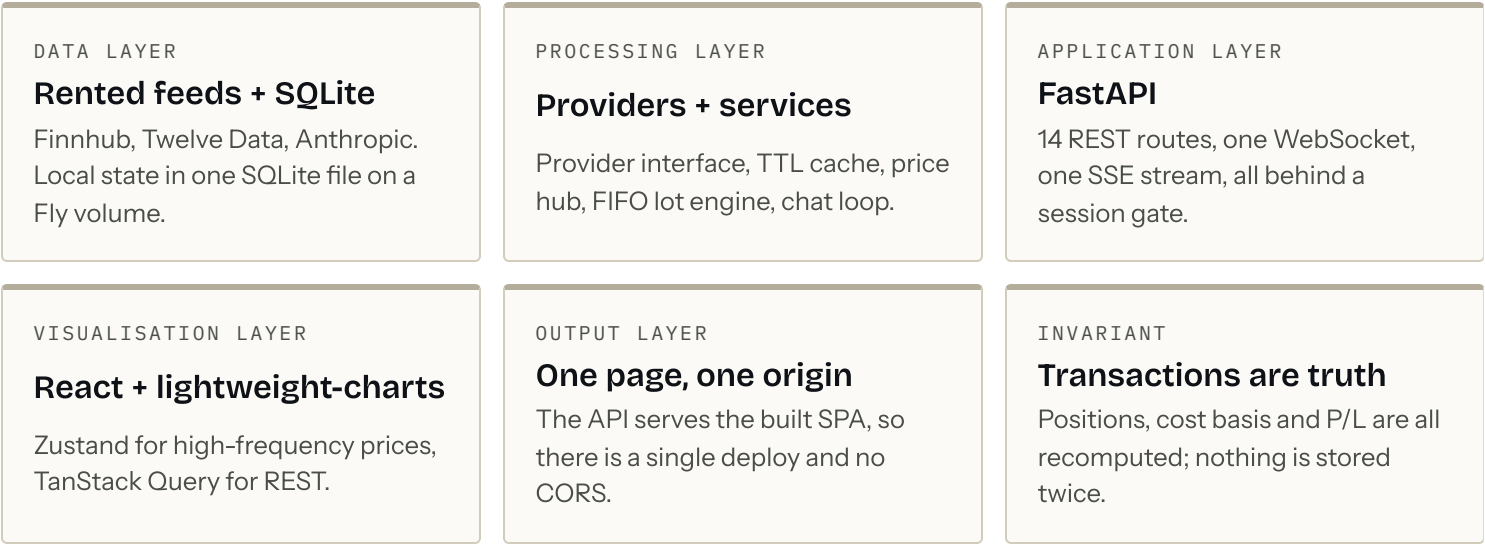
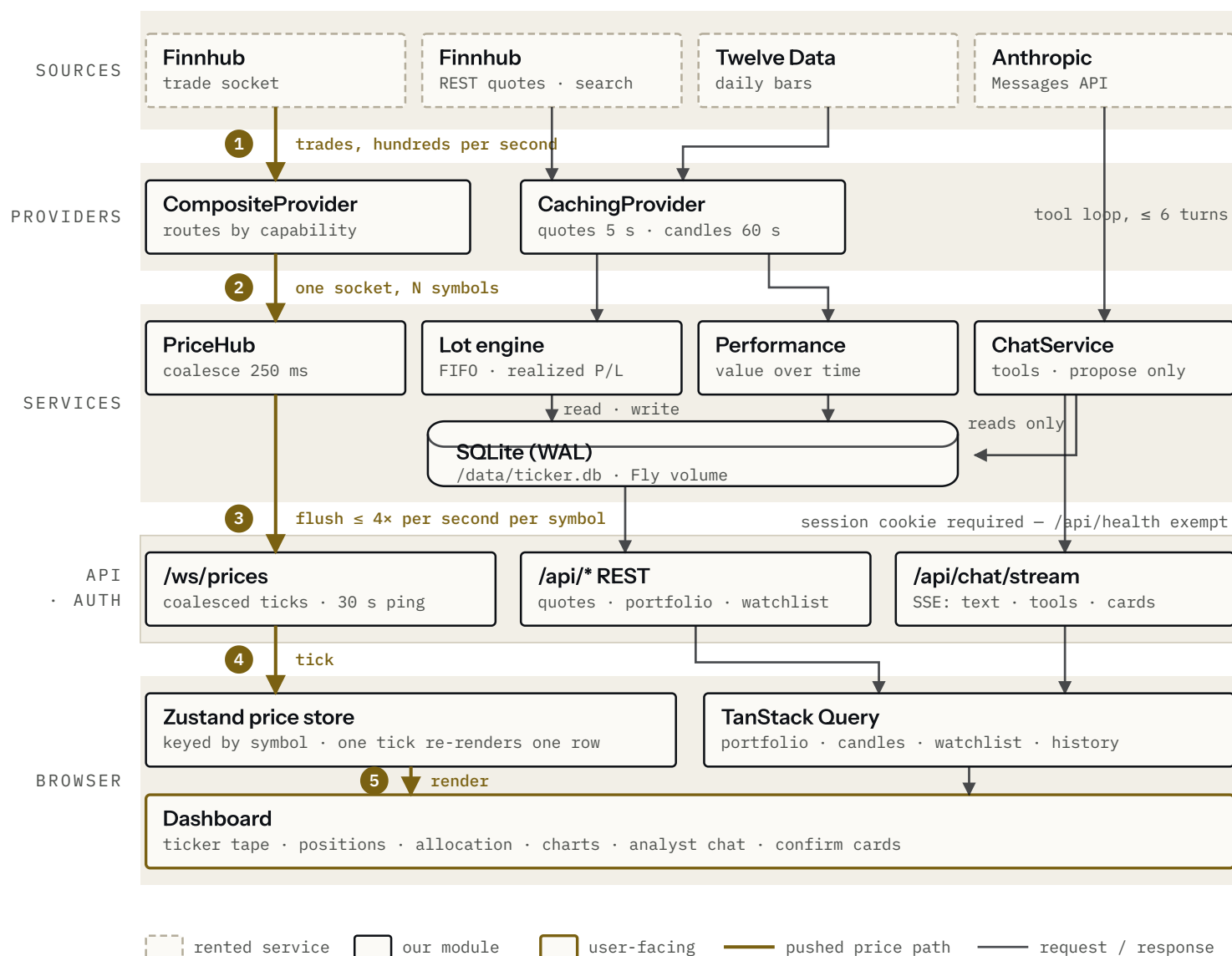


FIGURE 1 · RUNTIME ARCHITECTURE

How the system is layered, end to end

Five bands, read top to bottom. The gold path is a live price: the only route in the system that is pushed rather than requested, and the only one where a rate mismatch between source and browser has to be absorbed.

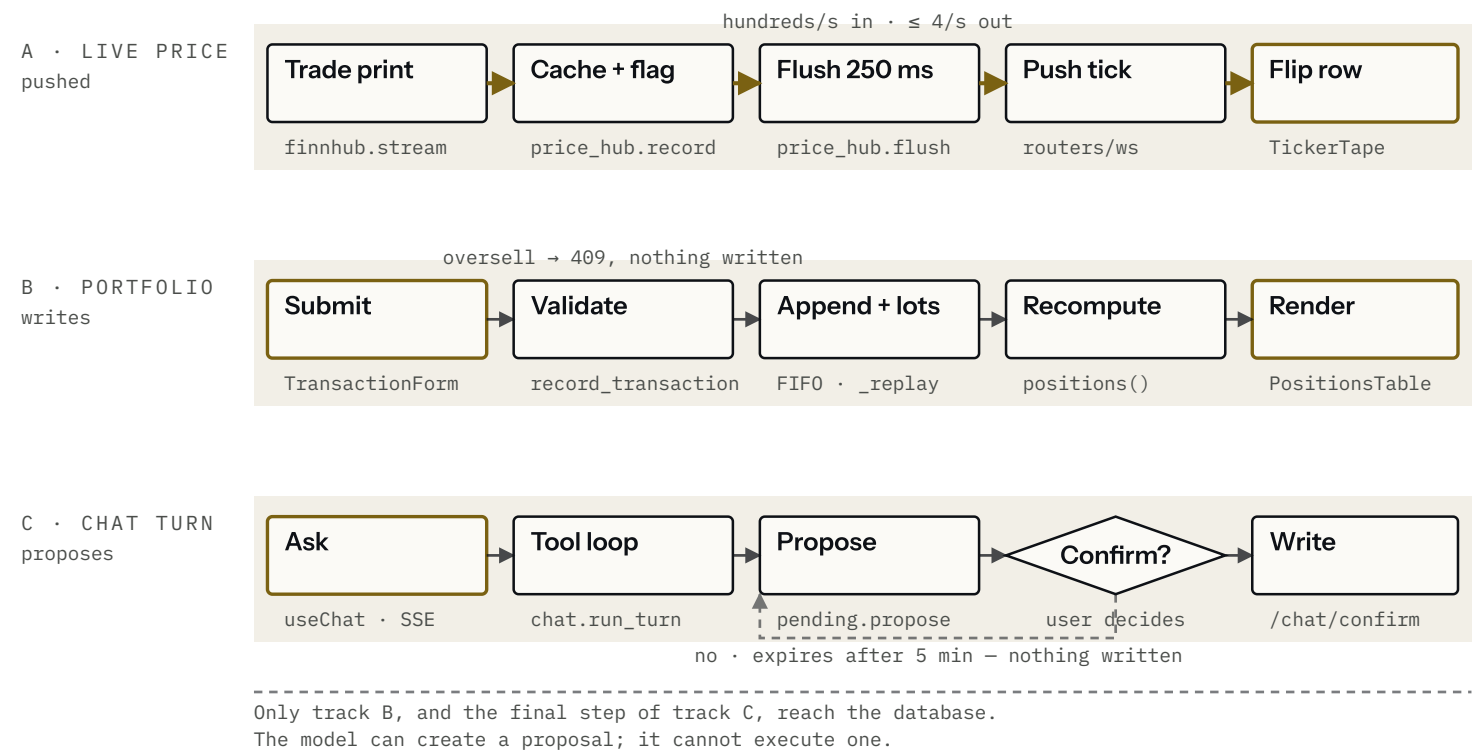


The rate mismatch is the design problem. A liquid symbol prints hundreds of trades a second (1); a browser cannot redraw that often. The PriceHub keeps the latest price per symbol and flushes at most every 250 ms (3), so the browser receives a bounded trickle. Anthropic bypasses the provider layer — it is not market data — and its tools read through the same services everything else uses, never writing. SQLite holds `transactions`, `lots`, `realized_pnl`, `watchlist` and `chat_messages`.

FIGURE 2 · PROCESS DETAIL

The three paths a change can take through the system

Everything the dashboard does is one of three flows. They share the same services but differ in what starts them and what they are allowed to touch: only the portfolio path and the final step of the chat path write to the ledger.



The tracks differ in what they may touch. Track A never sees the database. Track C's tool dispatcher is handed a read-only `PortfolioReader`, not a connection — it cannot write, because it holds no object capable of writing. The diamond is the only gate between a model proposal and a recorded trade.

STAGE REFERENCE

Every step, with its input, module and output

The same three tracks, written out. Each stage names the module responsible, so a failure anywhere in the dashboard can be traced to one file.

A1 A trade prints on the exchange

INPUT

Finnhub trade socket frame

PROCESS

Parse; skip malformed rows rather than dropping the feed

MODULE

`providers/finnhub.py · stream()`

OUTPUT

Trade objects, hundreds per second

A2 The hub absorbs the rate

INPUT

A Trade

PROCESS

Overwrite last price for the symbol; mark dirty. Deliberately does not send

MODULE

`services/price_hub.py · record()`

OUTPUT

In-memory cache + dirty set

A3 Flush, at most four times a second

INPUT

Dirty set, every 250 ms

PROCESS

Send the latest price per symbol to each subscriber that asked for it

MODULE

`services/price_hub.py · flush()`

OUTPUT

One tick frame per changed symbol

A4 Fan out to browsers, then render

INPUT

Tick frames on `/ws/prices`

PROCESS

Store keyed by symbol, so one tick re-renders one row

MODULE

`routers/ws.py → store/prices.ts → TickerTape.tsx`

OUTPUT

A digit flips on the tape

B1 A buy or sell is submitted

INPUT

Form fields, or a confirmed chat proposal

PROCESS

Pydantic validation; symbol normalised

MODULE

`routers/portfolio.py` · `POST /api/transactions`

OUTPUT

A validated `NewTransaction`

B2 Append to the ledger and rebuild lots

INPUT

`NewTransaction`

PROCESS

Reject oversell first; then append and consume oldest lots FIFO — one transaction, so a refusal leaves nothing behind

MODULE

`services/portfolio.py` · `record_transaction()`

OUTPUT

Rows in `transactions`, `lots`, `realized_pnl`

B3 Positions are computed, never stored

INPUT

Open lots

PROCESS

One grouped query, then priced with live quotes where available

MODULE

`services/portfolio.py` · `positions()`

OUTPUT

Quantity, cost basis, market value, unrealized P/L, allocation

B4 Value over time is reconstructed

INPUT	PROCESS	MODULE
Whole transaction log + daily closes	Replay forwards, valuing holdings at each day's close. A symbol with no history is named, never zero-filled	services/performance.py
OUTPUT		
Daily value + cost-basis series for the chart		

C1 A question is asked

INPUT	PROCESS	MODULE
Message text; rate limited to 20/hour	Load history, cap at ~20 turns, open an SSE stream	routers/chat.py · POST /api/chat/stream
OUTPUT		
Server-sent events: text deltas, tool activity, citations		

C2 The tool loop runs, read-only

INPUT	PROCESS	MODULE
Message + tool definitions	Up to 6 turns. The dispatcher holds a PortfolioReader, not a connection	services/chat.py · run_turn()
OUTPUT		
Quotes, positions, history, web results		

C3 **A change is proposed, not made**

INPUT	PROCESS	MODULE
propose_add_shares / propose_remove_shares	Validate symbol and quantity, fill price from the live quote if omitted, park in memory for 5 minutes	services/pending.py
OUTPUT		
A confirm card in the chat panel		

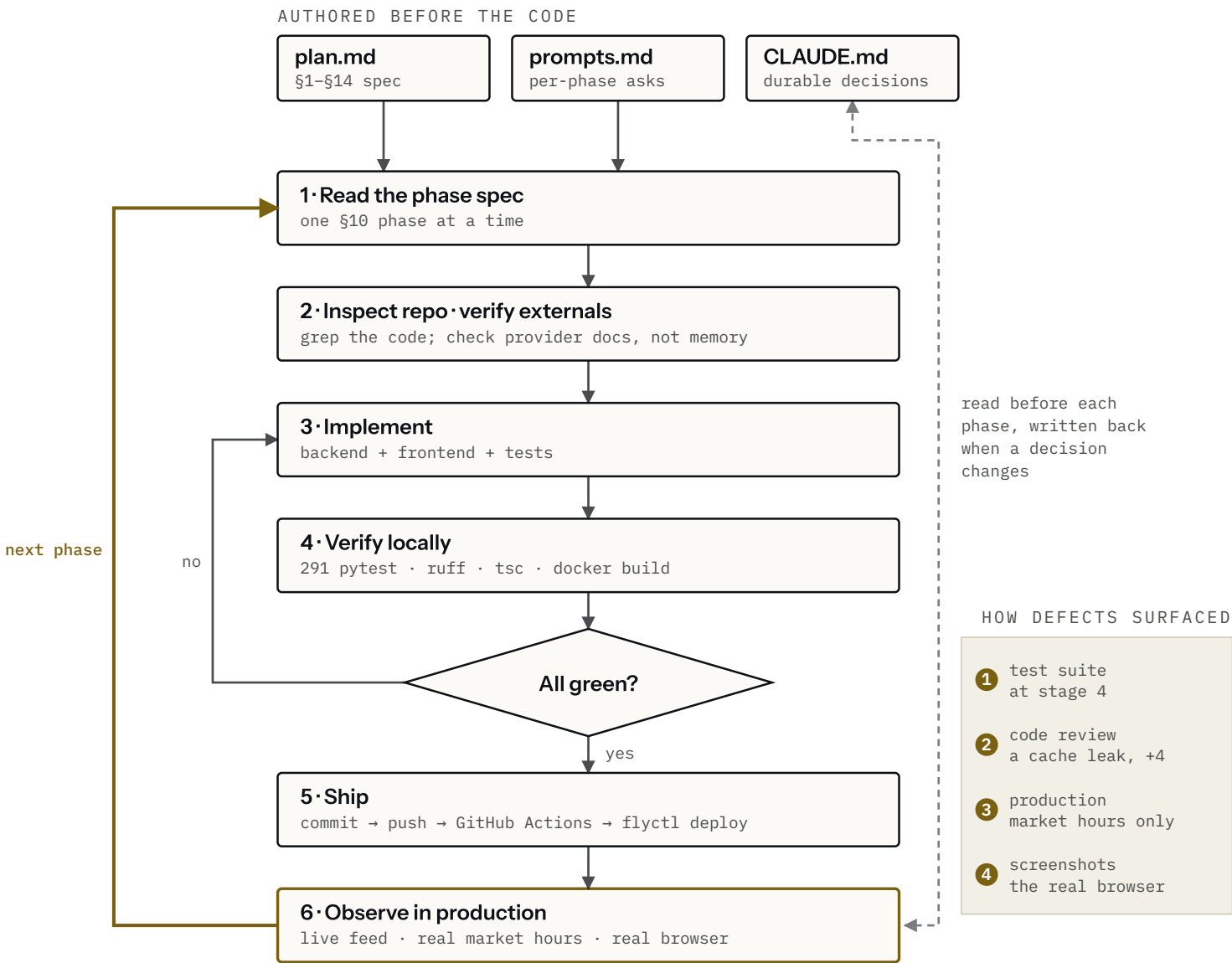
C4 **The user confirms — the only write path**

INPUT	PROCESS	MODULE
{action_id, approved}	Single-use: taking the proposal removes it, so a replayed confirm is a 410 and cannot double-book	routers/chat.py · POST /api/chat/confirm
OUTPUT		
Re-enters stage B2; the portfolio broadcasts to open tabs		

FIGURE 3 · DEVELOPMENT WORKFLOW

How the system was built: a spec-driven loop with four feedback channels

The project was written by an AI agent — Claude Code, running Claude Opus 5 — against a specification authored before any code existed. There is no autonomous task-runner here: each phase was a deliberate, human-initiated cycle. What makes the loop a loop is the four distinct ways a defect got discovered, three of which only fire after the code is already deployed.



Three of the four channels only fire after deploy. The test suite catches what can be checked in isolation. A review pass found an unbounded cache that every test passed. Production found a status update that was written but never called — invisible until a real market opened. A screenshot found a chart dying on a CSS colour token. Each produced a fix commit, and the git history interleaves them with the phase commits rather than hiding them.

The four outside services, and what each is for

Finnhub — the price feed

FREE TIER

DOES WHAT	Live prices and symbol search. Holds one permanently open connection to the market and pushes every trade through it.
WHY THIS ONE	The only free service offering genuinely real-time US prices over a push connection. Most free tiers are delayed, or make you poll.
THE CATCH	Its free tier stops at live prices. History is a paid feature — discovered only after deploying, when the request came back refused.
WITHOUT IT	No live prices. Cost basis and realized P/L are unaffected; they come from your own ledger.

Twelve Data — the history

FREE TIER

DOES WHAT	Past prices — the daily closes behind both charts. Nothing live.
WHY IT EXISTS	Purely to fill Finnhub's gap. Rather than pay for history, a second provider answers that one question, behind the same interface.
WHY THIS ONE	800 requests a day free. The obvious alternative allows 25 — six clicks on the chart range buttons would exhaust a day.
WITHOUT IT	Both charts return a 502 that names the fix. Everything else is unaffected.

Anthropic — the analyst

PAY PER USE

DOES WHAT	Powers the chat. It can read holdings, check prices, read transaction history and search the web, then answer in plain language.
THE IMPORTANT PART	It can propose portfolio changes but cannot make them. Nothing is written until you click Confirm.
WHY THAT MATTERS	Not distrust of the model. "Sell everything," typed at 2am, should meet a confirmation dialog like any other destructive action.
WITHOUT IT	The chat panel returns 503. The dashboard works normally.

Fly.io — where it runs

~\$3 / MONTH

DOES WHAT	Runs the container on one always-on machine with a public address and HTTPS. Hosting, not a component.
WHY THIS ONE	It keeps a long-lived connection open. Hosts that sleep between visits would kill the market socket every time you looked away.
ALSO PROVIDES	A volume — a disk that survives restarts. The database lives there; without it every deploy would wipe the ledger.
WITHOUT IT	The app runs on a laptop, reachable only by you, only while it is awake.

CONSTRAINTS

Five decisions that shaped everything else

Exactly one machine — for correctness, not cost

Two copies would open two connections to a feed that allows one, and each would keep its own database on a volume that cannot be double-mounted. The config pins it explicitly rather than relying on a default.

The transaction log is the only source of truth

Holdings, average cost and P/L are recomputed from the list of buys and sells every time. The log is append-only; a correction is a new row or a delete-and-rebuild. There is deliberately no stored share count, because average cost loses information the moment you sell part of a position.

The model proposes; the person disposes

Enforced structurally: the tool dispatcher receives a narrow read-only reader, never a database connection, so it cannot write. A test runs every tool and asserts the ledger is byte-for-byte unchanged.

It reports; it never advises

The system prompt forbids recommendations, price targets and predictions. Asked whether to sell, it declines and offers data. Every answer carries a disclaimer.

Degrade honestly, never guess

A price that is unavailable renders as an em dash, not a zero. Stale quotes are labelled with their age. A dead provider leaves market value null and names the symbols it could not price, rather than blanking the portfolio.

OPERATIONS

What it costs, and what keeps it alive

ITEM	WHAT IT PROVIDES	PER MONTH
Fly.io machine	Always-on server, public address, HTTPS	\$2-4
Fly.io volume	1 GB disk the ledger survives on	\$0.15
Finnhub	Live prices, 60 requests/min	\$0
Twelve Data	Price history, 800 requests/day	\$0
Anthropic	Chat, billed per message	a few \$
Typical total		under \$10

Migrations run at boot, idempotently — never as a release command, because that machine runs with no volume attached and the work would vanish. A nightly snapshot writes beside the live database using SQLite's online backup, not a file copy: with write-ahead logging, copying the database without its journal yields a torn file that looks fine until you need it.

STATUS

What is built

MARKET DATA

Live + historical

Streaming prices with honest staleness labels; daily bars behind both charts.

PORTFOLIO

FIFO ledger

Positions, cost basis, realized and unrealized P/L, allocation, value over time.

ANALYST

Tools + confirm

Reads holdings and the web; proposes changes you approve.

WATCHLIST

Editable tape

Add or remove tracked symbols; validated against the feed before saving.

SECURITY

Passphrase + limits

HttpOnly session cookie, rate-limited login, socket behind the same gate.

OPERATIONS

CI + backups

Deploy on every push, structured JSON logs, nightly database snapshot.

Not investment advice. This system reports data and answers questions about it. Prices may be delayed. Nothing in it is a recommendation to buy or sell anything, and it is built so that it cannot become one.